

1) What is Supabase?

- a) Supabase gives many backend features without having to build everything yourself. It is a open-source Firebase alternative.
- b) Supabase= Database + Authentication + API's+ File Storage + Realtime + other backend code.

2) Basic Idea

- a) For Example, in simple Web application:
 - i) React Frontend-> Express Backend -> Sequelize -> MySQL Database.
 - ii) We must build and manage: Express server, Api routes, authentication, JWT handling, db connection, db queries, file uploads, Realtime communication, permissions, etc.
- b) But with the Supabase many of the things are readymade services:
 - i) React->Supabase __> PostgreSQL DB, Authentication, Storage, Realtime, APIs, Security.
 - ii) Don't need to build express backend every feature.

3) Supabase include

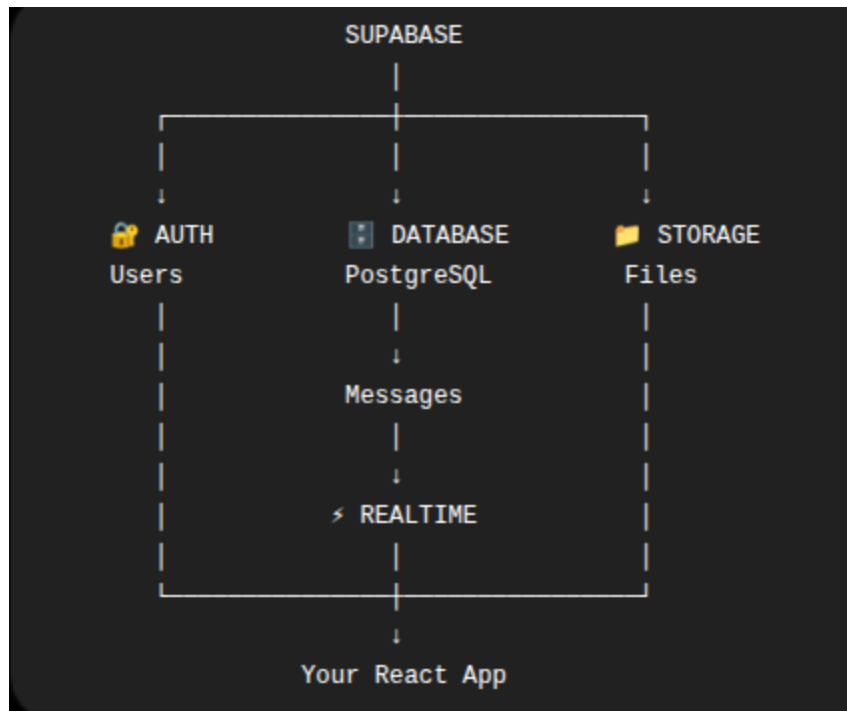
Supabase Feature	What it does
PostgreSQL Database	Stores your data
Authentication	Login/signup/users
Auto APIs	Gives APIs for database operations
Storage	Stores images/files/videos
Realtime	Sends database changes instantly
Row Level Security	Controls who can access what
Edge Functions	Runs backend/server-side code
Dashboard	Manage your project visually
Extensions	Adds PostgreSQL capabilities

4) Supabase Include Explanation:

Topic	Explanation
Dashboard	A visual place where we can manage the database, authentication, storage, Realtime, and Edge Functions.
Edge Functions	Allow us to write custom backend code. We can use them for sending emails, payments, webhooks, and other server-side tasks.
Realtime	Allows the application to receive updates immediately when data changes. It is useful for chat messages, notifications, and live updates.
Storage Buckets	Buckets are containers where we store files such as profile pictures, chat images, videos, and documents.
Storage	Used to upload and store files such as images, videos, and documents.
Database	Supabase provides a PostgreSQL database where we can store application data such as users, messages, and profiles.
AUTH	Supabase Auth supports common authentication methods such as email/password, magic links, OTP, social login, OAuth providers, and phone authentication. Who the user is?
API	Supabase automatically provides database APIs based on our tables. We can use the Supabase client directly without creating an API for every database operation in the backend. Flow: React → Supabase JS Client → Supabase API → PostgreSQL
RLS (Row Level Security)	Allows us to define rules that control which rows a user can read, insert, update, or delete. It controls which rows of a database table a user is allowed to access. Which data/rows can that user access?

Supabase

- |
- ├ Database → Store data
- ├ Auth → Login/Signup
- ├ API → Connect app with database
- ├ Storage → Store images/files
- ├ Realtime → Live updates
- ├ RLS → Control data access
- ├ Edge Functions → Custom backend code
- └ Dashboard → Manage everything



5) Row Level Security (RLS):

- a) RLS is a db feature that allows you to define policies restricting which row in table a user can: SELECT (read), INSERT (create), UPDATE (modify), DELETE (remove).
- b) It didn't return a row unless the policy allowed.
- c) RLS is useful when multiple users share the same tables but must see only their own rows.
- d) RLS policies act like invisible WHERE clauses applied by the db.
- e) The policy is evaluated per row, automatically, for every operation touching the tables. If a row fails, the policy -> it becomes invisible (SELECT) or rejected (WRITE).
- f) SELECT policy
- g) INSERT policy (WITH CHECK)
- h) UPDATE policy (USING + WITH CHECK)
- i) DELETE policy

6) Without RLS, how does MERN handle the problem that RLS Solve?

a) Express + Sequelize approach:

- i) React->/chat/20->protect route -> verify user -> Get authenticated userId -> WHERE senderId/receiverId -> MySQL

b) Supabase + RLS approach:

- i) React -> Supabase Auth-> PostgreSQL -> RLS policy-> only allowed rows.
- c) The goal is similar: Don't allow User A to access User B's private data.